



PROPOSAL REPORT
ON
**ACCESSICAP: BROWSER EXTENSION FOR EXCESSIBLE
WEB BROWSING**

SUBMITTED BY:

Sunib Bhakta Pradhananga

LUC No: LC00021000848

A Proposal Submitted to

Department of Computer Science and Multimedia

in the partial fulfillment of the requirements for the degree of

Bachelor of Information Technology (BIT)

at

National College of Management and

Technical Science

Samakhushi, Kathmandu

July, 2025

Kathmandu

Abstract

For individuals with visual impairments, reading problems, and other impairments, AccessiCap is an artificial intelligence-powered browser extension meant to improve web access. Real-time processing of web content, multiple language support, and thorough text-to-speech capabilities are provided together with automated descriptive alternative text for images created with artificial intelligence. AccessiCap enables the internet to be more accessible and inclusive without needing website changes by integrating easily with current browsers. This project shows how AI can really help make digital stuff more accessible, which is something we badly need. Using technology helps everyone to like internet content.

Web accessibility, AI alternative texts, inclusive digital experiences, visual impairment, multilingual support, text-to-speech, along with web browser extensions.

Acknowledgement

I want to show my project advisor my sincere thanks. Their advice, help, and thoughts were super important for this thing. Under no circumstances I could've completed excluding their assistance.

Big thanks to the Computer Science and Multimedia Department at Lincoln College for the resources that helped me get this done.

I give the Lincoln College Department of Computer Science and Multimedia thanks for giving me the resources and infrastructure needed to finish this project. Appreciation goes to the teaching personnel for their academic assistance and to my classmates for their developmental beneficial comments.

I recognize the work of the developers of the libraries and frameworks utilized in this project as well as the open-source community. The development and research in artificial intelligence access by companies like OpenAI, Hugging Face, and Google have been fundamental to this project.

Additionally, a big thank you to my friends and family for providing such a supportive atmosphere and understanding throughout my studying period and dedicating myself to this project.

Sunib Bhakta Pradhananga

LC00021000848

Preface

Created as a Final Year Project for the bachelor's degree in BIT at Lincoln College, "AccessiCap: Browser Extension for Accessible Web Browsing" is entirely documented here. The project shows how artificial intelligence may be applied in assistive technology and answers the rising need for affordable web technologies.

The documentation adheres to a systematic approach including every area of the project from start to execution. Included are technological specifications, design choices, implementation information, and results from testing. This paper is a technical reference for upcoming development and refinement of accessibility tools as well as a scholarly record.

While offering a useful, practical instrument for those with visual impairments and reading challenges, the project seeks to help the field of web accessibility.

Table of Contents

Abstract	I
Acknowledgement.....	II
Preface	III
List of Tables	VI
List of Figures	VII
Abbreviations	VIII
1. Introduction.....	1
1.1 Background of the Study.....	1
1.2 Problem Statement	1
1.3 Objective of the Project	1
1.4 Scope of the Project	2
1.5 Significance of the Project	2
1.6 Social Impact	3
1.7 Beneficiaries	3
2. Literature Review	4
2.1 WebAIM's WAVE Toolbar	4
2.2 Microsoft's Seeing AI	4
2.3 NVDA Screen Reader	4
2.4 Google's Lookout	5
2.5 axe Accessibility Checker: Developer-Focused Testing.....	5
2.6 Cultural Significance of AccessiCap.....	5
2.3 Research Gap / Innovation Justification.....	6
3. System Analysis and Design.....	7
3.1 Feasibility Study	7
3.1.1 Technical Feasibility	7
3.1.2 Operational Feasibility.....	7
3.1.3 Economic Feasibility	7
3.1.4 Legal and Ethical Feasibility	8
3.1.5 Schedule Feasibility	8
3.2 Requirements Specification.....	9
3.3 System Design	10

3.3.1 System Architecture.....	10
3.3.2 Use Case Diagram	11
3.3.3 Data Flow Diagram level 0	13
3.3.4 Data Flow Diagram level 1	13
3.3.5 Entity-Relationship Diagram	14
3.3.6 Sequence Diagram.....	15
3.3.7 Class Diagram	16
3.3.8 Database Schema.....	17
3.3.9 UI/UX Wireframes	18
3.3.10 Technology Stack.....	19
4. IMPLEMENTATION PLAN	20
4.1 Development Methodology	20
4.2 Module Breakdown.....	20
4.3 Project Timeline (Gantt Chart).....	21
4.4 Risk Analysis	23
5. Current Progress.....	24
6. EXPECTED OUTCOMES AND LIMITATIONS	26
6.1 Expected Product Features.....	26
6.2 Limitations.....	27
7. CONCLUSION	27
7.1 Summary of the Proposed Word.....	27
7.2 Future Scopes	28
8. REFERENCES	29
9. Appendices	31

List of Tables

Table 1: Comparative Analysis of Existing System.....	6
Table 2: Functional Requirement Specification.....	9
Table 3: Non-Functional Requirements Specification	10
Table 4: Technology Stack	21
Table 5: Risk Analysis	23
Table 6: Expected Features List	26

List of Figures

Figure 1: System Architecture	11
Figure 2: Use Case Diagram	12
Figure 3: Data Flow Diagram lvl 0	13
Figure 4: Data Flow Diagram lvl 1	13
Figure 5: Entity-Relationship Diagram	14
Figure 6: Sequence Diagram	15
Figure 7: Class Diagram	16
Figure 8: Database Schema	17
Figure 9: UI/UX Wireframes	18
Figure 10: Technology Stack	19
Figure 11: Gantt Chart	22
Figure 12: scanning pop up	31
Figure 13: UI/UX 2	31
Figure 14: UI/UX	31
Figure 15: customizable TTS settings	31
Figure 16: Settings	31
Figure 17: Page scanner and TTS activation shortcut and Tutorial	32
Figure 18: Accessibility Features options	32

Abbreviations

AI - Artificial Intelligence

API - Application Programming Interface

BLIP - Bootstrapping Language-Image Pre-training

CORS - Cross-Origin Resource Sharing

DFD - Data Flow Diagram

ERD - Entity-Relationship Diagram

HTML - HyperText Markup Language

JSON - JavaScript Object Notation

ML - Machine Learning

TTS - Text-to-Speech

UI - User Interface

UX - User Experience

WCAG - Web Content Accessibility Guidelines

CSS - Cascading Style Sheets

JS - JavaScript

IDE - Integrated Development Environment

HTTP – Hypertext Transfer Protocol

1. Introduction

1.1 Background of the Study

Accessibility on websites is now a key priority in the present day. According to the WHO, 285 million people throughout the world experience difficulties with eyesight. Notwithstanding contemporary changes, they have trouble seeing things on the internet. Pictures ought to have captions, per the Web Content Accessibility Guidelines (WCAG) yet, several websites leave these out. In most cases, users need to create these summaries manually, resulting in time consumption and is not consistently executed effectively. New artificial intelligence, especially in how machines see and understand language, can help fix this automatically. The BLIP model from Salesforce Research is great at writing accurate captions for images. Also, browser extensions can now change web pages in complex ways. This project uses these tools to make a practical, real-time solution for accessibility that doesn't need any changes to websites.

1.2 Problem Statement

Individuals who are visually impaired often struggle with online images when a proper caption is absent clarifying their content. Currently, the existing support mechanisms are insufficient **because** the user needs to manually create descriptions, they lack proper support for multiple languages moreover browsers, automatically generated descriptions are sometimes inaccurate, and they are unable to adapt to real-time changes. This issue poses a challenge for individuals with visual impairments to access the in the digital sphere world and violates the principle that all individuals possess unbiased access online.

1.3 Objective of the Project

The primary objective of this project is to develop an AI-powered browser extension that automatically generates descriptive captions for web images and provides comprehensive accessibility features for visually impaired users. Specific objectives include implementing real-time AI image captioning with 95% accuracy

using the BLIP model, supporting twelve languages with integrated translation, offering customizable accessibility settings, providing text-to-speech functionality, ensuring compatibility with major browsers, designing an intuitive WCAG-compliant interface, and building a scalable backend architecture.

1.4 Scope of the Project

The AccessiCap project is all about making web browsing easier. It's creating a browser extension for Chrome and Edge that uses AI to add captions to images and tweak other settings to help folks out.

It works with most image types you find online, whether the page is still or moving. Plus, it speaks twelve languages and has a bunch of accessibility options you can mess with. It's mainly for people who have trouble seeing, have issues with reading, older people, those learning a new language, or anyone who just wants easier web browsing.

Right now, it won't work on phones, with videos, PDFs, social media, or big business setups.

Target Users

- **Visually Impaired Users:** Individuals with partial or complete vision loss
- **Users with Reading Difficulties:** People with dyslexia, ADHD, or cognitive disabilities
- **Elderly Users:** Older adults experiencing age-related vision changes
- **Language Learners:** Users of accessing content in non-native languages
- **General Users:** Anyone preferring enhanced accessibility features

1.5 Significance of the Project

This project really matters academically, socially, and technically. For school, it pushes AI research for accessibility and gives a good look at how browser

extensions are built. It's also great socially because it helps blind people get online easier, cuts down on needing to manually fix accessibility issues, and helps different language groups get access. On the tech side, it puts AI models to work, shows how to do full-stack development with all the newest stuff, and gives a blueprint for making accessibility tools.

1.6 Social Impact

For people with reading difficulties or eyesight issues, AccessiCap makes websites much more user-friendly. Making everyone able to get online and view the same material is everything about this. It increases prospects for disabled people to learn and work by removing those obstacles. This instrument also encourages developers and corporations to consider inclusive design, or a more accessible internet for everyone. Moreover, it runs in several languages, hence it is quite useful in areas when usually English tools are the only choice.

1.7 Beneficiaries

- Visually challenged: This encompasses individuals with complete or partial blindness. Different methods of internet viewing are required.
- Devices that read text out loud or facilitate reading can assist those with dyslexia, ADHD, or other learning problems.
- Elderly people: Our eyes can alter as we age. Improved vision capabilities can enable elderly people to consume information more readily.
- Language Learners: Learning a new language especially benefits from translations and pronunciation assistance.
- Everyone Else: Although you only want to know more about how to simplify websites, access features can be really beneficial for anybody.

2. Literature Review

2.1 WebAIM's WAVE Toolbar

WebAIM's WAVE Toolbar is a browser extension that checks if the website is accessible for disabled or not. According to the guidelines of WCAG 2.1 and Section 508 (WebAIM, 2025; Web Standards Commission, 2024), it looks for problems like incorrect color contrast, poor alt text in images, and some coding errors found in a webpage. Experts say that while WAVE primarily helps locate issues by displaying them visually, fixing them calls for manual intervention. It's not easy to use on websites with a lot of different languages or that change a lot because it only works in English and doesn't have any artificial intelligence. Despite these limitations, WAVE is a well-liked tool for accessibility instruction and expert reviews since it's a simple approach to learn about and verify accessibility (WebAIM, 2025).

2.2 Microsoft's Seeing AI

Seeing AI is a mobile application developed by Microsoft which assists blind individuals or are visually impaired. It uses computer vision and machine learning to ID objects, text, and people. Studies show it's pretty accurate at recognizing things in the real world, which helps users be more independent (Microsoft Accessibility Blog, 2023; IxD@Pratt, 2024). But, it only works on mobile, not on web browsers. This limits how much it can help with web accessibility. Experts see Seeing AI as a helpful tool for getting around in the physical world, but it doesn't do enough for digital inclusion on the web (Responsible AI Toolkit, 2024).

2.3 NVDA Screen Reader

Many people think that the free, open-source screen reader NVDA is a good substitute for paid programmes like JAWS (Job Access With Speech). Research reveals that it helps to simplify the use of digital material, especially in academic settings (Gaikwad et al., 2025; Kirboyun Tipi, 2023). Although NVDA works well with text-to-speech capability, new users may find its installation challenging. It's best for text as it can't automatically describe images as it's not AI-based. Though experts point out NVDA's worth in equitable access, they also caution that it's not fantastic with anything outside of text (Varghese & Rathnasabapathy, 2020).

2.4 Google's Lookout

Using artificial intelligence, the Android app Google Lookout can identify objects and read text. Studies show that it is user-friendly and reliable, which enables those with vision impairment to move around. But since Lookout is constrained to Android and lacks web browsing capabilities, it cannot help much with digital accessibility. Researchers find that Lookout is among a few artificial intelligence technologies good for physical mobility but not for web access.

2.5 axe Accessibility Checker: Developer-Focused Testing

Deque's axe Accessibility Checker is a tool that developers commonly use for automatic testing of their websites' accessibility. It combines technologies such as Google Lighthouse to detect issues that are not compliant with WCAG standards (Deque, 2024). Studies point out that it is a trusted and indispensable tool for professional accessibility evaluations. But then again, proper Axe use demands a bit of technical know-how as it cannot be relied on to eradicate problems by itself (Apify, 2024; W3C WAI, 2024). According to the professionals, Axe is one step closer to automated accessibility testing, but its effectiveness will largely depend on the skills of the developers and the commitment of the entire organization to the cause of accessibility.

2.6 Cultural Significance of AccessiCap

AccessiCap is all about digital inclusion, mirroring a cultural move to make info and tech accessible to everyone. In South Asia, community support and caring for each other are big things. AccessiCap lets people with vision and reading issues use web content on their own, which lines up with these values of helping people, respecting them, and getting them involved in society. The project changes the web from being all about visuals to a place where everyone can participate, no matter what. This thing doesn't just fix a tech problem, it gets people thinking about accessibility, pushing developers and groups to focus on design that includes everyone.

Table 1: Comparative Analysis of Existing System

2.7 Comparative Analysis

Feature	WAVE	Seeing AI	NVDA	AccessiCap
Platform	Browser	Mobile	Desktop	Browser
AI Integration	No	Yes	No	Yes
Real-time Processing	No	Yes	No	Yes
Multilingual Support	No	Limited	No	Yes (12 languages)
Automatic Alt-text	No	Partial	No	Yes
Accessibility Modes	No	No	Limited	Comprehensive
User Experience	Technical	Good	Complex	Intuitive
Integration	Manual	None	System-level	Seamless

2.3 Research Gap / Innovation Justification

Key points from our research have been the main issues that AccessiCap is tackling. Put simply, one of the biggest problems that are raised is the tools that are currently available for users to access web content are not properly interconnected as they are just different applications instead of being parts of a browser. Besides that, until now, no one has thought of the idea to use AI for providing in real, time a description of a web image. In addition, nearly all the tools are designed only for English speakers thus a user who speaks another language might face some difficulties. Lastly, in most cases, existing devices have been programmed in such a way that

they emphasize only one feature of accessibility thereby leaving the consumers with partial solutions instead of comprehensive ones. AddressiCap would fix these problems by giving users a browser solution that is integrated, employing AI in real time, accommodating more languages, and enabling users to have full access to all the features of the web.

3. System Analysis and Design

3.1 Feasibility Study

3.1.1 Technical Feasibility

The project is practicable due to its reliance on mature, well-documented technologies: browser extension tools for accessibility evaluation, the BLIP model for automated image annotation, and Python FastAPI for backend services. Each component is actively maintained and backed by an engaged developer community, providing robust documentation, frequent updates, and community support that together mitigate technical risk and facilitate sustainable development.

3.1.2 Operational Feasibility

AccessiCap is easy to use and doesn't need training. It has low system demands, works with current websites without needing changes, and has an easy-to-understand, accessible design. It fits easily into browsing habits.

3.1.3 Economic Feasibility

By using open-source tech, we avoid license payments. Cloud-based services is not required when scaling. Maintaining functionality doesn't require a large budget, and there aren't any individual subscriptions, thereby ensuring it's within reach for individuals and organizations.

3.1.4 Legal and Ethical Feasibility

This system meets the requirements of WCAG 2.1 principles, follows webpage policies, maintains private records locally in order to protect privacy, additionally utilizes licenses with publicly available source code so the entire system is publicly viewable. The project sticks to user freedom and data safety ethics.

3.1.5 Schedule Feasibility

A four-month timeframe to build this is possible. We plan to use Agile methodologies for sustained progress, a design based on modules that enable team members to work concurrently, in addition, we plan to reuse pre-existing code to minimize the initial development timeline. That way the structure of the project is organized makes it

3.2 Requirements Specification

Table 2: Functional Requirement Specification

Requirement ID	Description
FR-01	System shall automatically detect and process images on web pages
FR-02	System shall generate accurate image descriptions using AI
FR-03	System shall support 12 language translations
FR-04	System shall provide text-to-speech for image descriptions
FR-05	System shall offer multiple accessibility modes
FR-06	System shall maintain user preferences
FR-07	System shall provide context menu options
FR-08	System shall support keyboard shortcuts
FR-09	System shall track usage statistics
FR-10	System shall check backend server status

Table 3: Non-Functional Requirements Specification

Requirement Type	Description
Performance	Image processing within 5 seconds, TTS latency < 1 second
Usability	Intuitive interface, WCAG 2.1 compliant, 90% user satisfaction
Reliability	99% uptime for backend, graceful degradation when offline
Security	Local data storage, no personal data collection
Compatibility	Chrome 88+, Edge 88+, Windows 10+, macOS 10.15+
Maintainability	Modular code, comprehensive documentation
Scalability	Support up to 1000 concurrent users
Accessibility	Fully accessible to visually impaired users

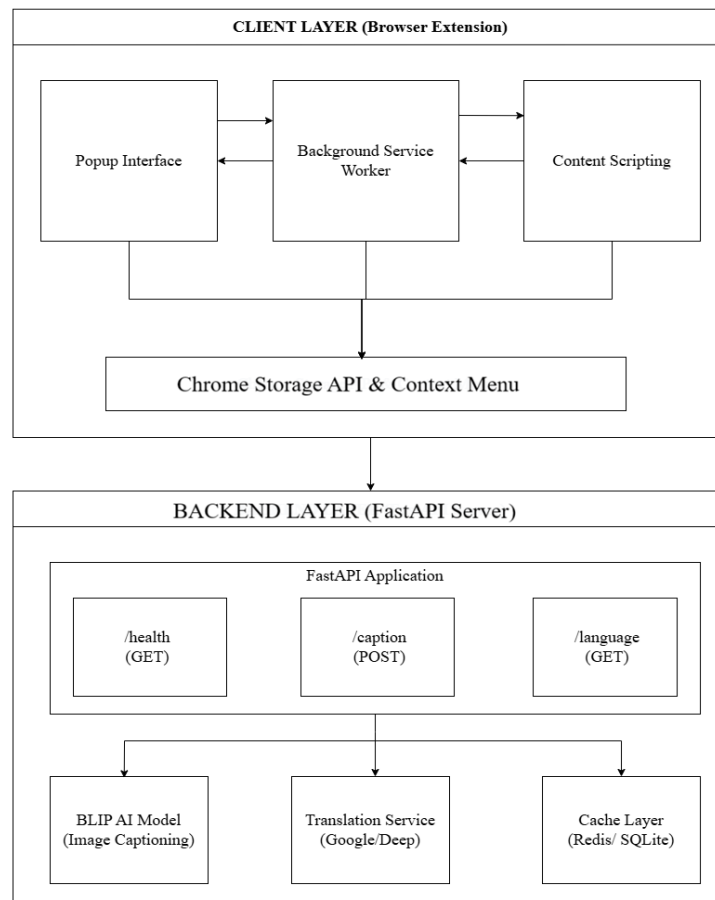
3.3 System Design

3.3.1 System Architecture

The system follows a client-server architecture with three main components:

- Browser Extension (Frontend)
- Backend API Server
- AI Model Service

Figure 1: System Architecture



3.3.2 Use Case Diagram

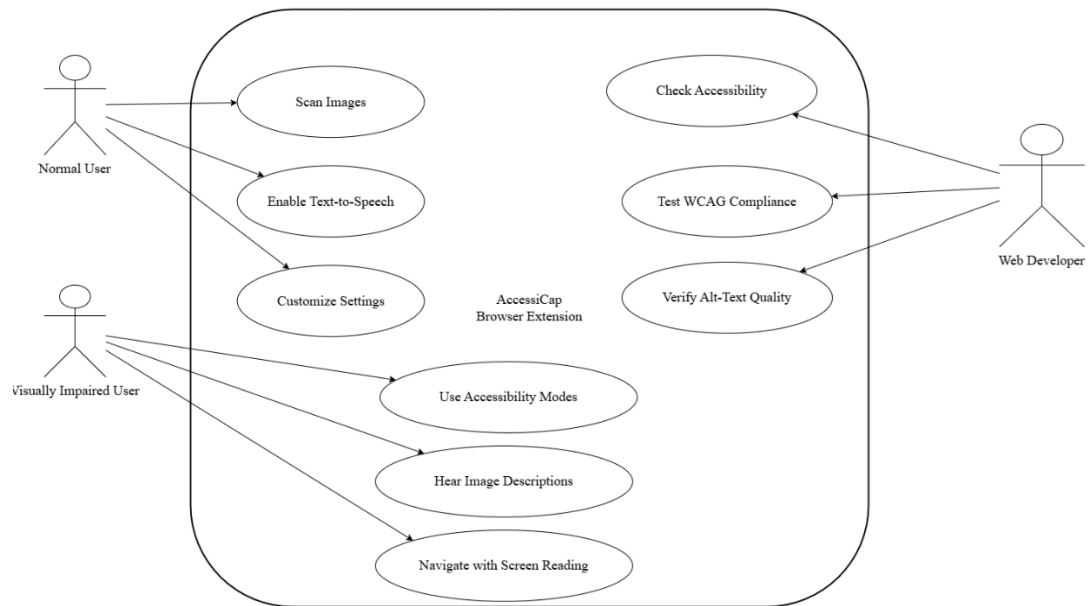
Primary actors:

User, Browser, AI, Server

Main use cases:

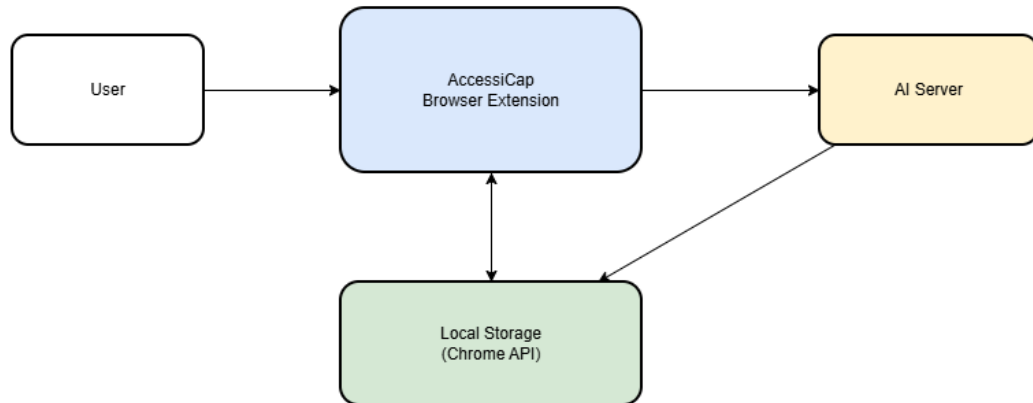
- Browse website
- Process images
- Read descriptions
- Adjust settings

Figure 2: Use Case Diagram



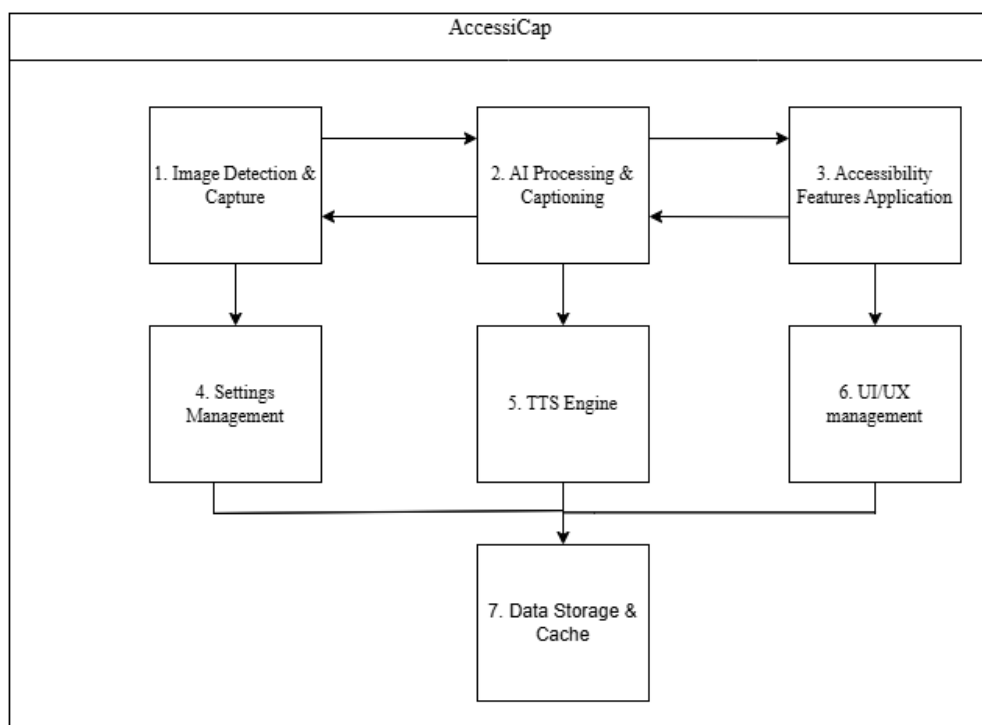
3.3.3 Data Flow Diagram level 0

Figure 3: Data Flow Diagram lvl 0



3.3.4 Data Flow Diagram level 1

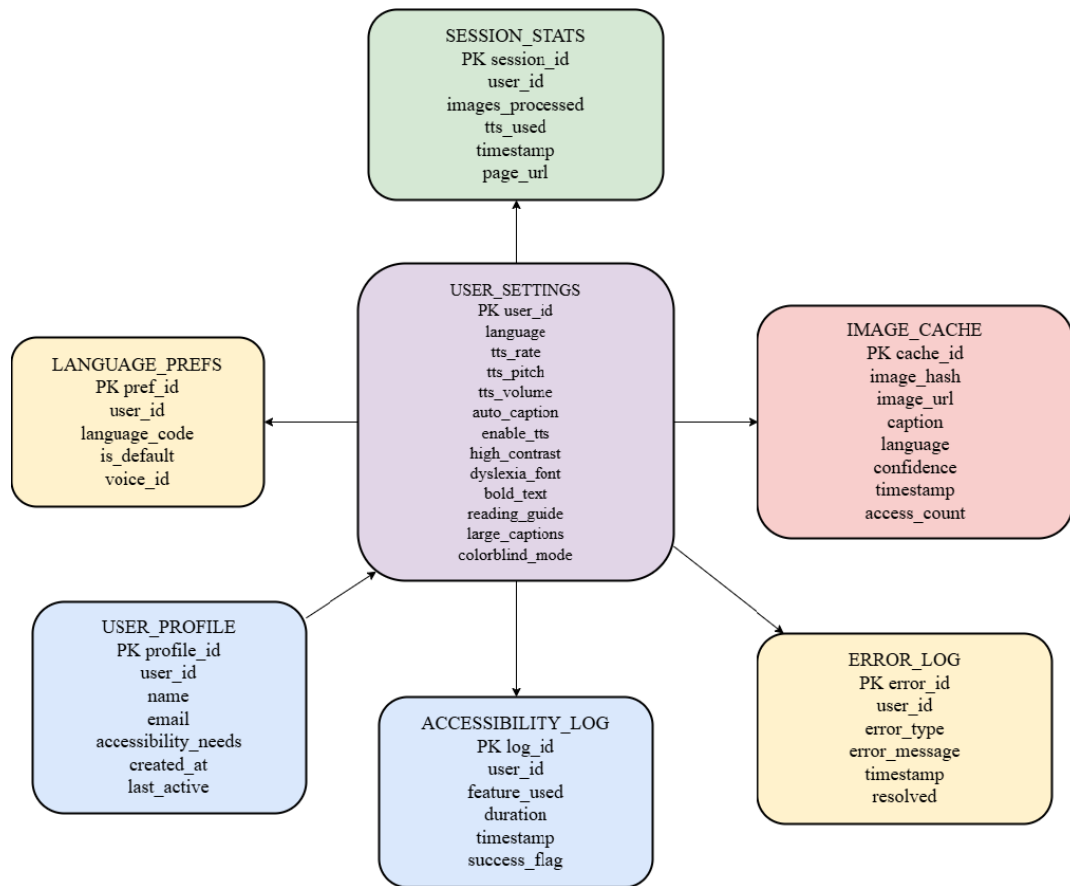
Figure 4: Data Flow Diagram lvl 1



3.3.5 Entity-Relationship Diagram

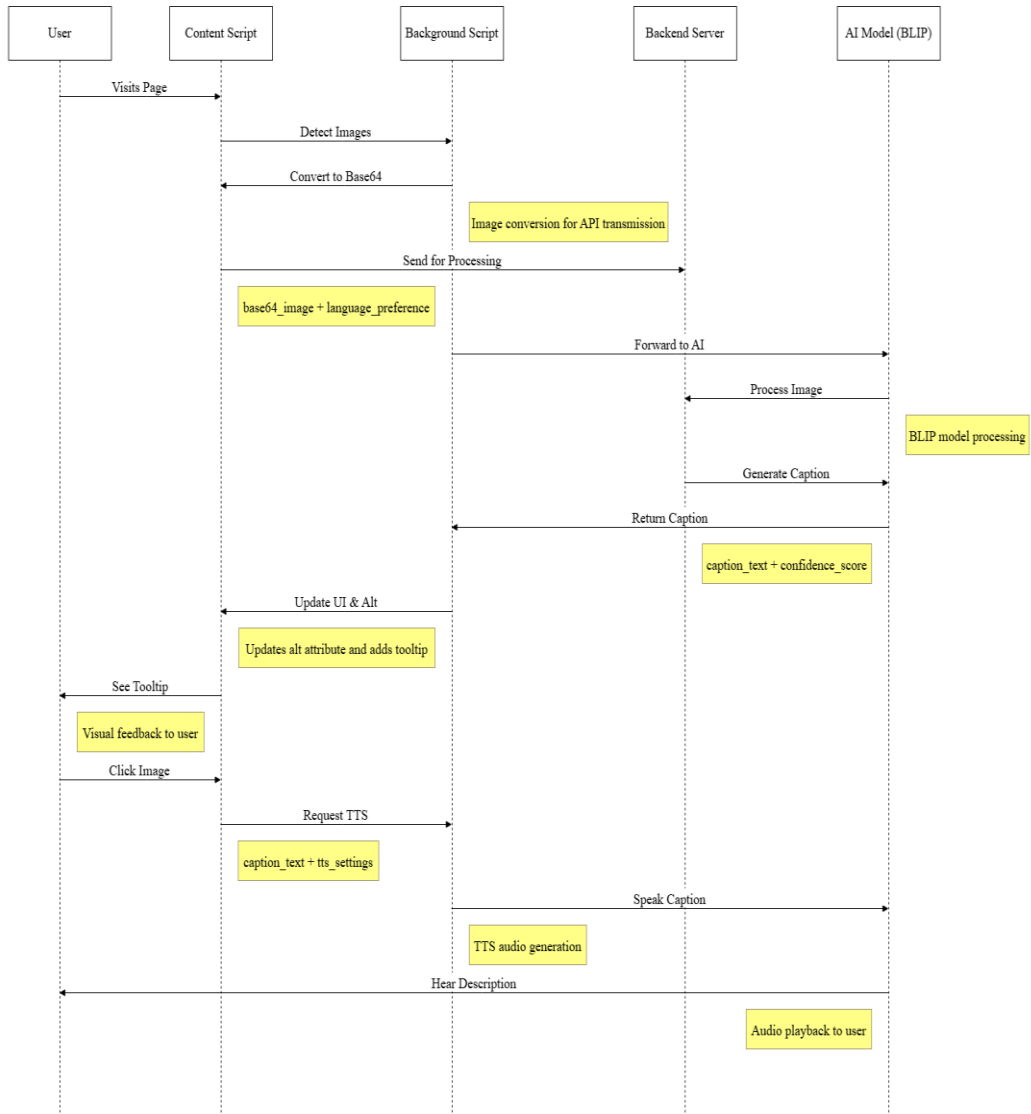
Entities: User Settings, Session Data, Image Cache, Language Preferences

Figure 5: Entity-Relationship Diagram



3.3.6 Sequence Diagram

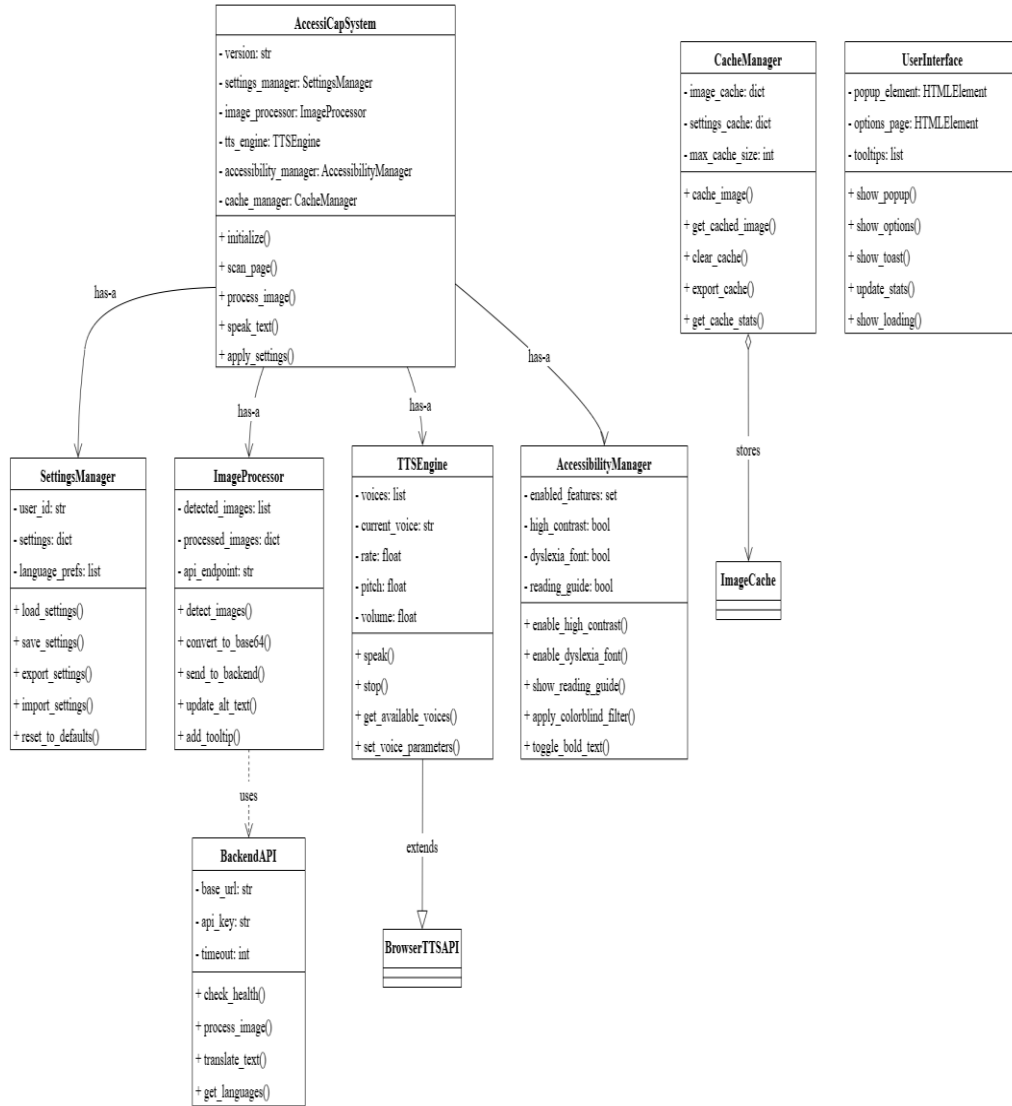
Figure 6: Sequence Diagram



3.3.7 Class Diagram

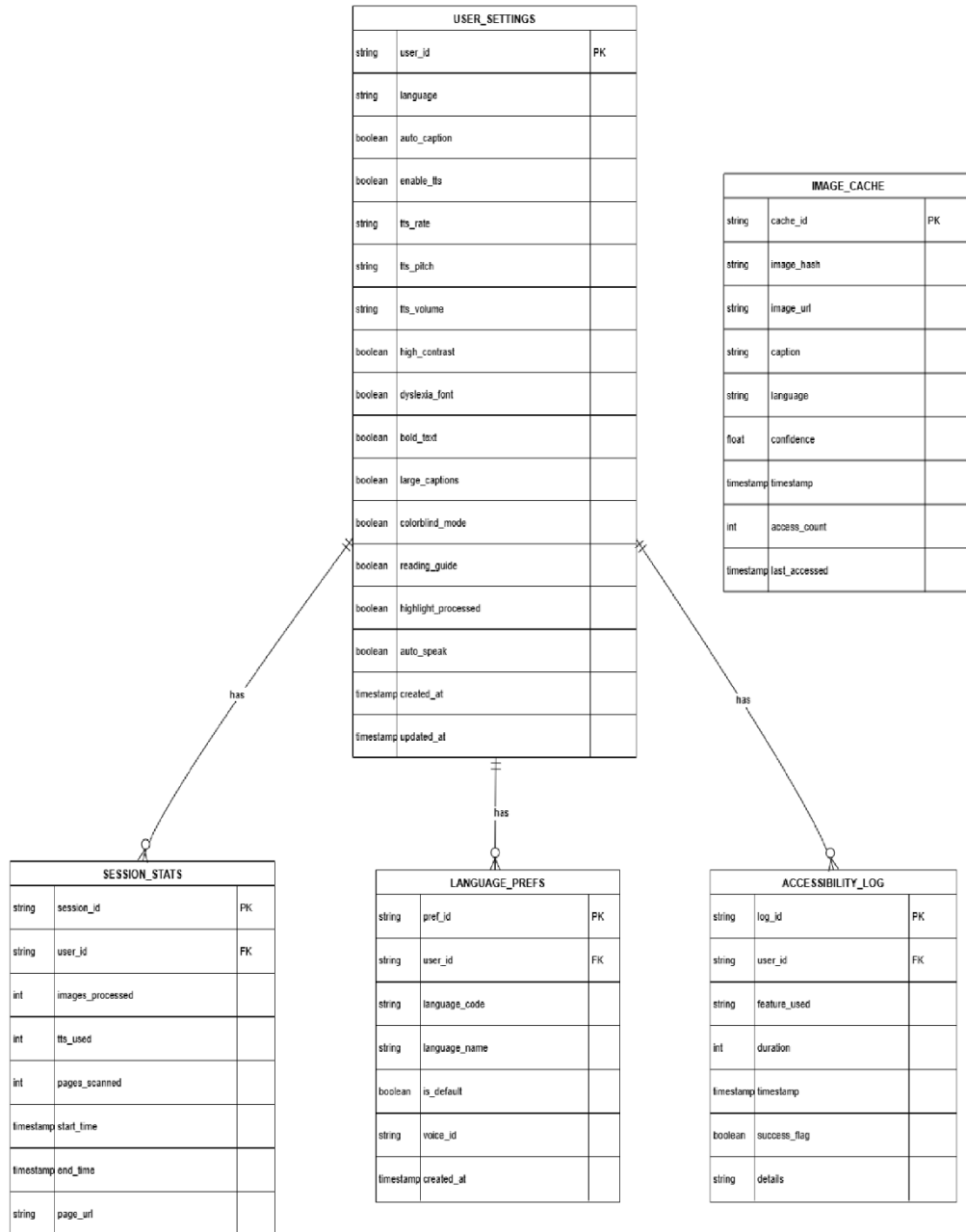
Main classes: ImageProcessor, TTSEngine, SettingsManager, LanguageHandler, AccessibilityManager

Figure 7: Class Diagram



3.3.8 Database Schema

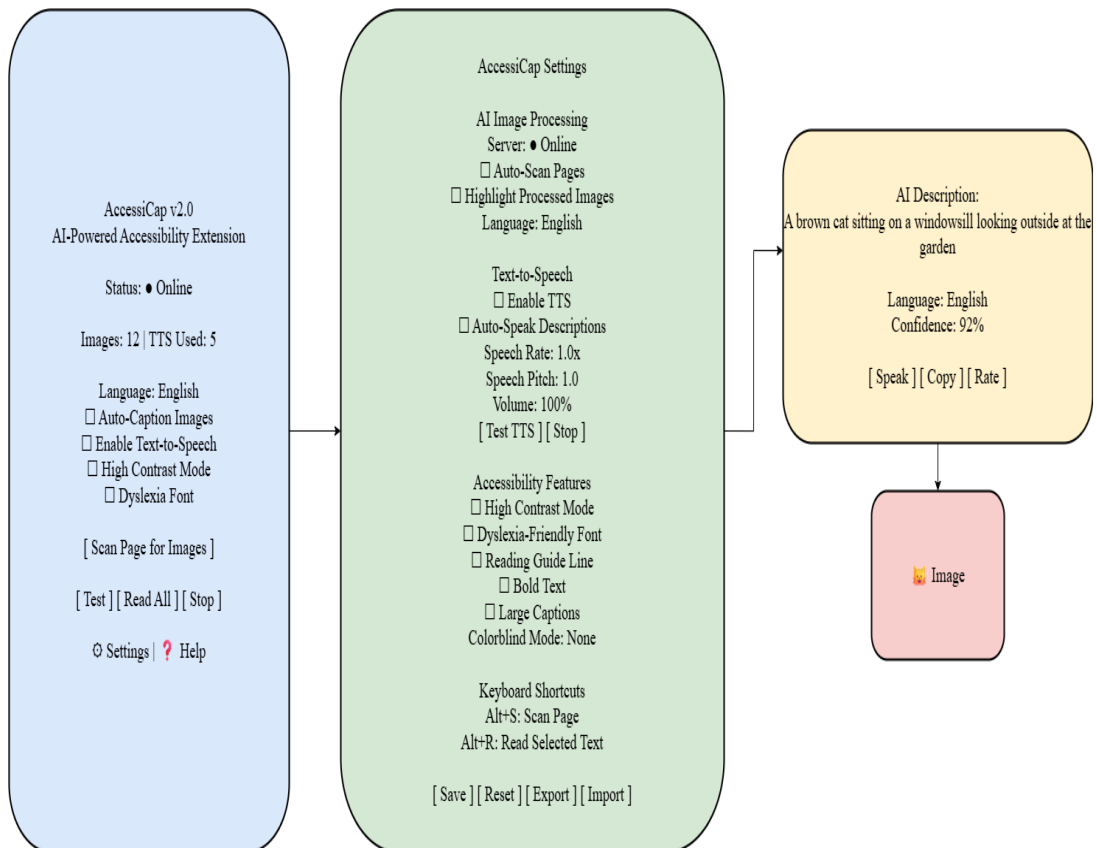
Figure 8: Database Schema



3.3.9 UI/UX Wireframes

- Popup interface with status, controls, settings
- Options page with comprehensive settings
- Tooltip design for image descriptions
- Loading states and error handling

Figure 9: UI/UX Wireframes

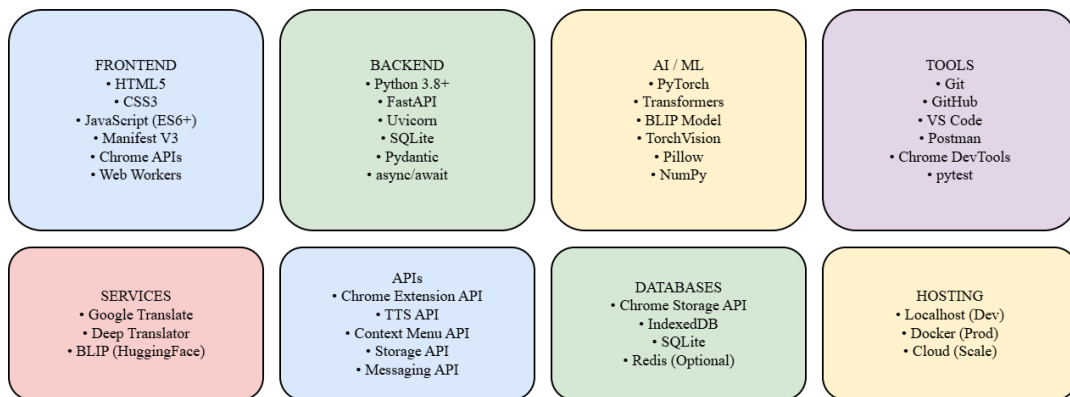


3.3.10 Technology Stack

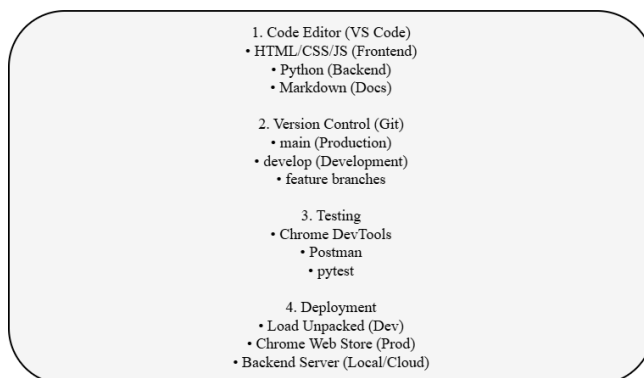
- **Frontend:** HTML5, CSS3, JavaScript ES6+, Manifest V3
- **Backend:** Python 3.8+, FastAPI, Uvicorn
- **AI/ML:** PyTorch, Transformers, BLIP model
- **Translation:** Google Translate API, deep-translator
- **Database:** Chrome Storage API (local), SQLite (backend)
- **Development:** Git, VS Code, Chrome DevTools

Figure 10: Technology Stack

TECHNOLOGY STACK ARCHITECTURE



DEVELOPMENT WORKFLOW



4. IMPLEMENTATION PLAN

4.1 Development Methodology

We are operating with Agile Scrum and two, week sprints. It's adaptable, and we can do the steps over again. Thus, we can handle the complex AI stuff, get user feedback on the fly, detect and correct errors quickly, and keep testing and integrating everything. Our method is sprint planning, daily stand, ups, sprint reviews, and retrospectives.

4.2 Module Breakdown

- a) **Browser Extension Module:** Content scripts for image detection, background scripts for message routing and TTS, popup interface for user controls, and options page for settings management.
 - b) **AI Processing Module:** Incorporates BLIP model for image captioning, is language translation capable, and has implemented cache management for performance optimization.
 - c) **Accessibility Module:** Offers visual modes (high contrast, colorblind filters), accessibility features (dyslexia font, reading guide), and TTS engine for text, to, speech functionality.
 - d) **Backend Server Module:** Designed with FastAPI to serve endpoints, manage AI models, and keep track of system health.
- #### 4.3 Tools, Platforms, and Languages

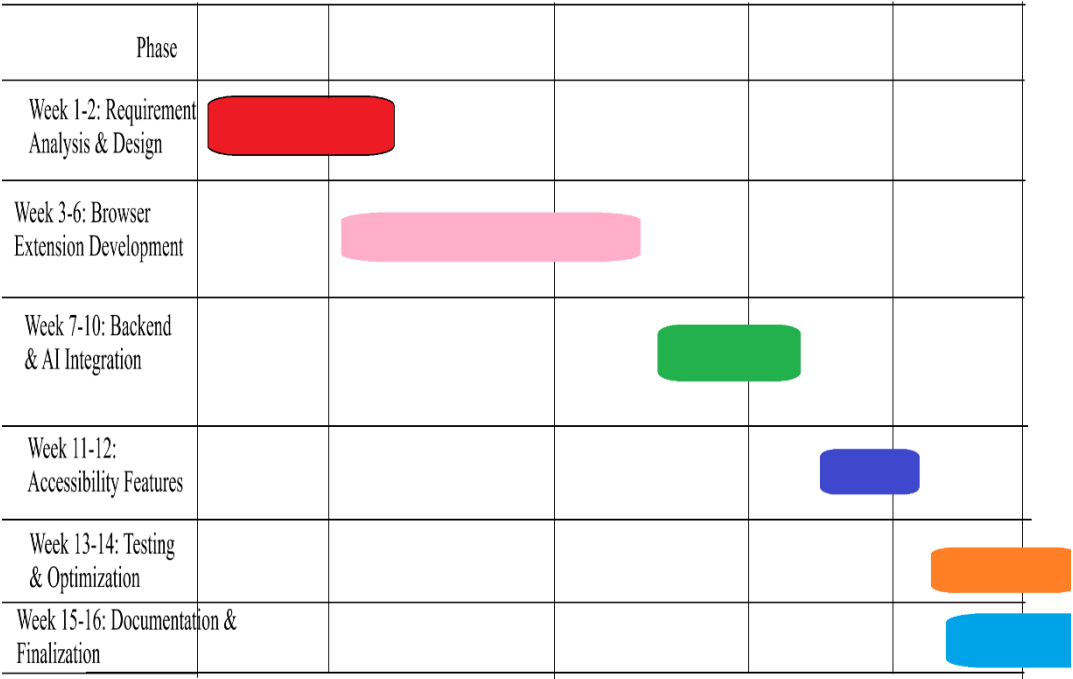
Table 4: Technology Stack

Category	Tools & Technologies
Frontend Development	HTML5, CSS3, JavaScript ES6+, Chrome Extension APIs
Backend Development	Python 3.8, FastAPI, Uvicorn, PyTorch
AI/ML Frameworks	Transformers, BLIP model, TorchVision
Database	Chrome Storage API, SQLite
Version Control	Git, GitHub
IDE	Visual Studio Code, Jupyter Notebook
Testing	Chrome DevTools, Postman, pytest
Documentation	LaTeX, Markdown, Doxygen
Deployment	Local server, Docker (optional)

4.3 Project Timeline (Gantt Chart)

- Weeks 1-2: Requirement Analysis and Design
- Weeks 3-6: Browser Extension Development
- Weeks 7-10: Backend and AI Integration
- Weeks 11-12: Accessibility Features
- Weeks 13-14: Testing and Optimization
- Weeks 15-16: Documentation and Finalization

Figure 11: Gantt Chart



4.4 Risk Analysis

Table 5: Risk Analysis

Risk Description	Likelihood	Severity	Impact	Mitigation Strategy
AI model accuracy issues	Medium	High	High	Implement fallback captions, user feedback system
Browser API limitations	Low	Medium	Medium	Use polyfills, alternative approaches
Translation API failures	Medium	High	High	Multiple translation providers, caching
Performance bottlenecks	High	Medium	Medium	Implement caching, optimize image processing
Cross-browser compatibility	Medium	High	High	Extensive testing, progressive enhancement
User adoption barriers	Medium	Medium	Medium	Intuitive design,

				comprehensive documentation
Legal/Privacy concerns	Low	High	High	Local data storage, transparent data policies
Time constraints	High	Medium	Medium	Agile methodology, prioritization, MVP approach
Technical expertise gaps	Medium	High	High	Training, documentation, peer collaboration

5. Current Progress

As of the mid-defense stage, the AccessiCap project has achieved substantial implementation of core functionalities. The AI-powered browser extension successfully processes images, generates captions, and provides accessibility features, though refinement and compatibility improvements are ongoing. As of now it has successfully implemented these features:

- Browser extension framework using Manifest V3 with service workers, content scripts, and popup interface.
- Context menu integration and keyboard shortcuts (Alt+S for scan, Alt+R for read)

- AI image processing pipeline with BLIP model integration for real-time captioning
- Multilingual support for 12 languages with translation and TTS
- Text-to-speech engine with customizable rate, pitch, and volume controls
- Accessibility features including high contrast mode, dyslexia font, colorblind filters, and reading guide
- Backend FastAPI server running on localhost:8001 with CORS support
- User settings management with Chrome Storage API and export/import functionality
- Cross-browser compatibility for Chrome 88+ and Microsoft Edge 88+
Basic testing across multiple website types and performance benchmarking

6. EXPECTED OUTCOMES AND LIMITATIONS

6.1 Expected Product Features

Table 6: Expected Features List

Feature Category	Specific Features
Image Processing	Automatic detection, AI captioning, real-time processing
Multilingual Support	12 languages, translation, language detection
Text-to-Speech	Image description reading, page text reading, selection reading
Accessibility Modes	High contrast, dyslexia font, colorblind filters, bold text
User Interface	Popup controls, options page, context menus, tooltips
Integration	Keyboard shortcuts, browser integration, seamless operation
Customization	Settings export/import, preference management
Performance	Caching, optimization, minimal resource usage

AccessiCap should give image captions in real time with 95% rightness. It works in 12 languages, changes text to speech, and has several accessibility modes. It's simple to use, works with browsers without problems, and can be changed how you like. The goal is to cut down how much alt-text work people do by 90%, assist over 285 million users with sight problems around the world, and be able to understand images in less than 5 seconds. It is designed to promote online inclusion, grow accessibility understanding, and push for inclusive web design.

6.2 Limitations

Technical limitations include support for only common image formats, dependency on internet connectivity for AI processing, initial restriction to Chrome/Edge browsers, inability to process video or complex media, and significant backend resource requirements. Functional limitations involve potential AI misinterpretation of complex images, loss of cultural nuance in translations, challenges with rapidly changing dynamic content, inability to process authenticated images, and scalability constraints. Deployment limitations encompass the need for regular updates, lack of centralized management for enterprise use, and desktop-only support initially.

7. CONCLUSION

7.1 Summary of the Proposed Work

Combining advanced artificial intelligence with useful browser extension development, AccessiCap is a major step forward in web accessibility technology that tackles a serious digital inclusion problem. The project's integrated approach solves the accessibility of web photos by needing no site changes and offers instant advantages for end-users.

The suggested system shows how millions of people could have access to sophisticated computer vision capabilities by means of transformer-based artificial intelligence models practically employed for social benefit. The initiative solves varied user demands while keeping simplicity and user-friendliness by means of 12 languages and several accessibility options.

Using established technologies but also novel integrations, the technical approach balances creativity with utility. While the open-source method promotes community engagement and openness, modular architecture guarantees maintainability and extensibility.

7.2 Future Scopes

Short-term enhancements include extending support to Firefox and Safari, adding more languages, enabling video content processing, developing offline mode with lightweight models, and introducing enterprise deployment options. Medium-term goals involve mobile application versions, social media platform integration, advanced AI features, community captioning features, and analytics dashboards. Long-term vision includes automated WCAG compliance, live video captioning, VR/AR accessibility features, integration with other assistive technologies, and predictive accessibility based on user behavior.

8. REFERENCES

1. World Health Organization. (2020). World report on vision. WHO Press.
2. Li, J., Li, D., Xiong, C., & Hoi, S. (2022). BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation. Proceedings of the 39th International Conference on Machine Learning.
3. WebAIM. (2025). *WAVE Web Accessibility Evaluation Tools*. Retrieved from
4. Web Content Accessibility Guidelines (WCAG) 2.1. (2018). W3C Recommendation.
5. IxD@Pratt. (2024). *Assistive Technology: Microsoft Seeing AI*. Retrieved from <https://www.ixd.prattsi.org>
6. Clark, J., & Safari, O. (2003). Building Accessible Websites. New Riders Press.
7. Microsoft Corporation. (2021). AI for Accessibility: Annual Report. Microsoft Research
8. Gaikwad, S., Khan, A. A., & Deshpande, R. S. (2025). *A Functional Study of NVDA for Visually Impaired Learners in Digital Learning*. IJCRT
9. Kirboyun Tipi, S. (2023). *How Screen Readers Impact the Academic Work of College and Graduate Students with Visual Impairments*. Dergipark
10. Google Accessibility. (2022). Chrome Accessibility Features Documentation. Google Developer Documentation.
11. FastAPI Documentation. (2023). Official FastAPI Documentation. Retrieved from <https://fastapi.tiangolo.com>
12. PyTorch. (2023). PyTorch Documentation. Retrieved from <https://pytorch.org/docs>
13. Chrome Extensions Documentation. (2023). Chrome Developer Documentation. Retrieved from <https://developer.chrome.com/docs/extensions>
14. Farra, D. (2024). *Google's Lookout: Prioritizing Beauty and Accessibility in App Design*. Google Design

14. Vaswani, A., et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems* 30.
15. Brown, T., et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* 33.
16. Thatcher, J., et al. (2006). *Web Accessibility: Web Standards and Regulatory Compliance*. Apress.
17. W3C Web Accessibility Initiative. (2021). *Accessibility Principles*. Retrieved from <https://www.w3.org/WAI/fundamentals/accessibility-principles>
18. United Nations. (2006). *Convention on the Rights of Persons with Disabilities*. United Nations.
19. Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann.
20. United Nations. (2006). *Convention on the Rights of Persons with Disabilities*. United Nations.
21. Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann.

9. Appendices

Figure 14: UI/UX

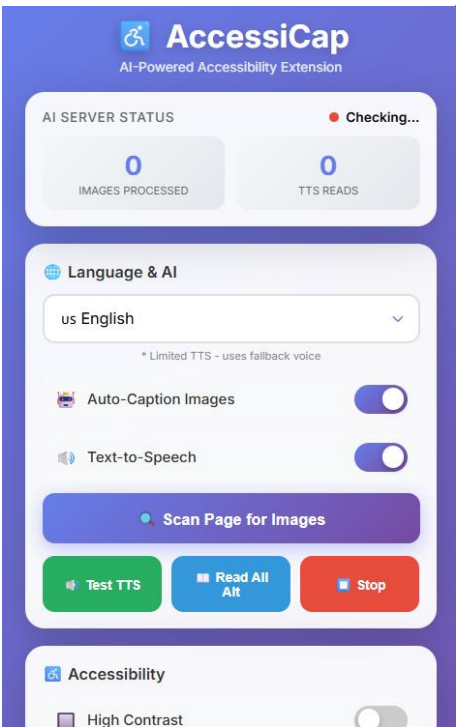


Figure 13: UI/UX 2

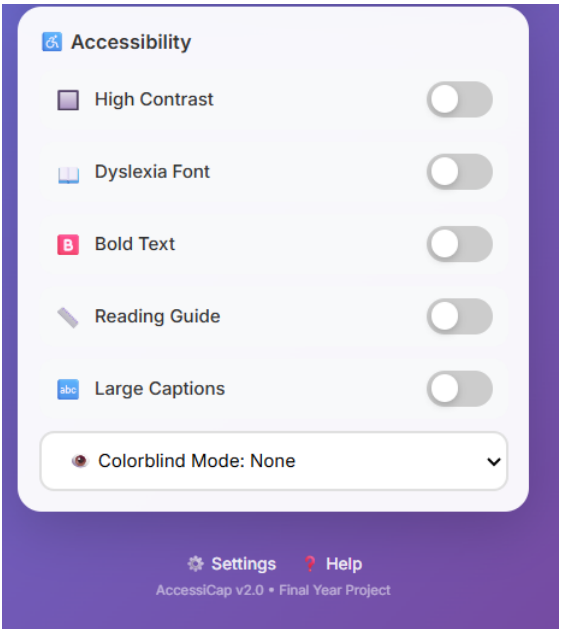


Figure 12: scanning pop up

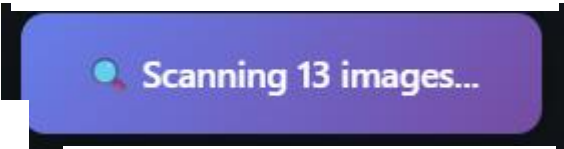


Figure 15: customizable TTS settings

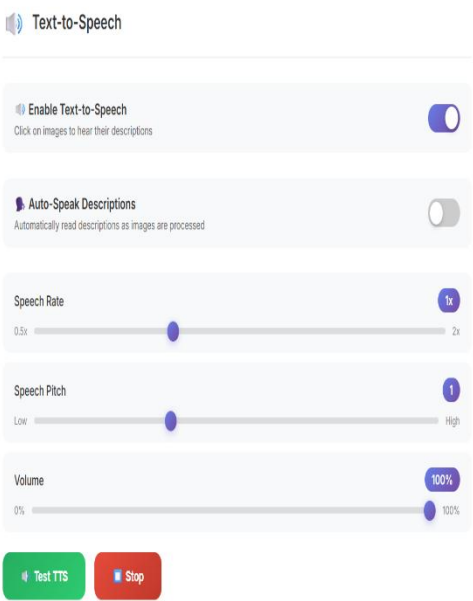
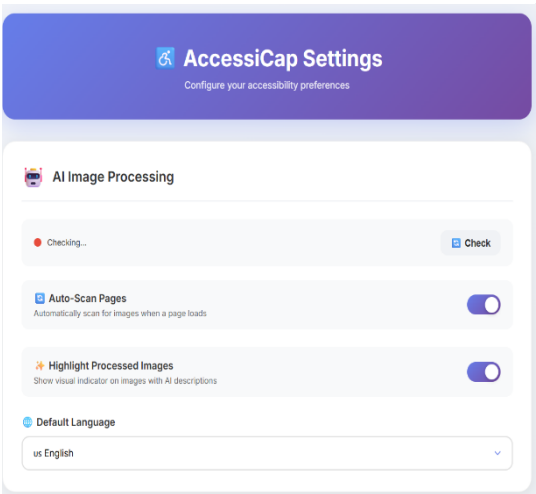


Figure 18: Accessibility Features options

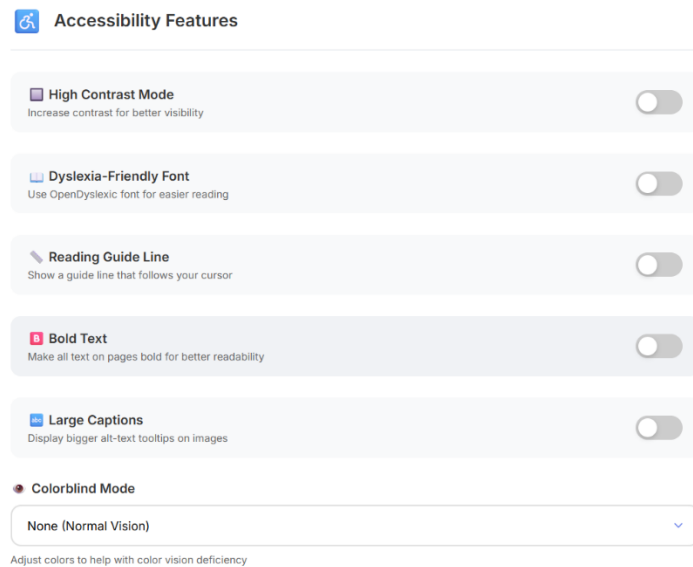


Figure 17: Page scanner and TTS activation shortcut and Tutorial

